

AstroBurst

High-Performance Astronomical Image Processor

Technical Document — Release v0.3.0

Samuel Krieger Bonini

March 2026

Field	Value
Document Version	0.3.0
Release	v0.3.0
Date	March 2026
Author	Samuel Krieger Bonini
Stack	Rust 1.75+ · Tauri v2 · React · TypeScript
Platforms	Windows · macOS · Linux
License	GPL-3.0-or-later
Repository	https://github.com/samuelkriegerbonini-dev/AstroBurst

Contents

1	Executive Summary	4
1.1	What's New in v0.3.0	4
2	System Architecture	4
2.1	Backend (Rust)	5
2.2	Frontend (React + TypeScript)	5
2.3	IPC Strategy	6
3	Mathematical Foundations	6
3.1	Screen Transfer Function (STF)	6
3.2	Asinh Stretch	7
3.3	Sigma-Clipped Stacking	7
3.4	Drizzle Integration	8
3.5	Cross-Correlation Alignment	8
3.6	World Coordinate System (WCS)	9

3.6.1	Forward Transform (Pixel $\rightarrow$ World)	9
3.6.2	Pixel Scale	9
3.6.3	Inverse Transform (World $\rightarrow$ Pixel)	9
3.7	FFT Power Spectrum	9
3.8	Star Detection (PSF Centroid)	10
3.9	SCNR (Subtractive Chromatic Noise Reduction)	10
3.10	Calibration Pipeline	10
3.11	White Balance	10
3.12	Richardson-Lucy Deconvolution (v0.3.0)	11
3.13	Background Extraction (v0.3.0)	11
3.14	Wavelet Denoising (v0.3.0)	11
4	Command Module Reference	12
4.1	Image Module (<code>commands/image.rs</code>)	13
4.2	Metadata Module (<code>commands/metadata.rs</code>)	13
4.3	Analysis Module (<code>commands/analysis.rs</code>)	13
4.4	Visualization Module (<code>commands/visualization.rs</code>)	14
4.5	Cube Module (<code>commands/cube.rs</code>)	14
4.6	Astrometry Module (<code>commands/astrometry.rs</code>)	14
4.7	Stacking Module (<code>commands/stacking.rs</code>)	14
4.8	Config Module (<code>commands/config.rs</code>)	15
4.9	Processing Module (<code>commands/processing.rs</code>) — New in v0.3.0	15
5	FITS I/O Implementation	15
5.1	Memory-Mapped Extraction	15
5.2	Supported BITPIX Values	16
6	ASDF I/O Implementation (v0.3.0)	16
6.1	Format Structure	16
6.2	Decompression Support	16
6.3	Roman Data Model Traversal	16
6.4	gWCS Extraction	16
6.5	Auto-Dispatch	16
7	WebGPU Compute Pipeline	17
8	Performance Benchmarks	17

9 Dependencies	18
9.1 Rust Crates	18
9.2 Frontend	18
10 Security Considerations	18
11 Known Limitations	19
12 Roadmap	19
A Sample Test Data	19

1 Executive Summary

AstroBurst is a cross-platform desktop application for processing, analyzing, and visualizing astronomical FITS and ASDF images. It is the first astronomical image processor built on the **Rust/Tauri/WebGPU** stack, combining native-level performance with a modern, accessible user interface.

The application targets two user segments:

- Professional astronomers working with observatory data (JWST, Hubble, Roman Space Telescope, ground-based IFU instruments)
- Advanced astrophotographers processing deep-sky narrowband and broadband imaging data

Version 0.3.0 delivers a complete processing pipeline from raw FITS/ASDF ingestion through deconvolution, noise reduction, color composition, and export, with **41 backend commands** exposed through a typed TypeScript API layer. Of these, 20 commands are actively wired to the frontend UI, while 21 additional commands are backend-ready and awaiting UI panel integration.

1.1 What's New in v0.3.0

- **Richardson-Lucy deconvolution** — FFT-based iterative deconvolution with Gaussian PSF, regularization, and deringing
- **Background extraction** — Polynomial surface modeling with sigma-clipped grid sampling (subtract or divide modes)
- **Wavelet denoising** — Multi-scale wavelet decomposition with per-scale sigma thresholds (up to 8 scales)
- **ASDF format support** — First Rust implementation of the Advanced Scientific Data Format reader with zlib/bzip2/lz4 decompression, Roman Space Telescope data model traversal, and gWCS extraction
- **Redesigned tab UI** — Bottom tabs (Info, Analysis, Headers, Export) + side panels (Processing, Compose) with resizable drag handles
- **Split preview contexts** — 6 independent React contexts for minimal re-renders
- **Lazy-loaded panels** — React.lazy + Suspense for fast initial load

2 System Architecture

AstroBurst follows a strict backend/frontend separation, with all computation offloaded to Rust and all rendering handled by the browser engine embedded in Tauri.

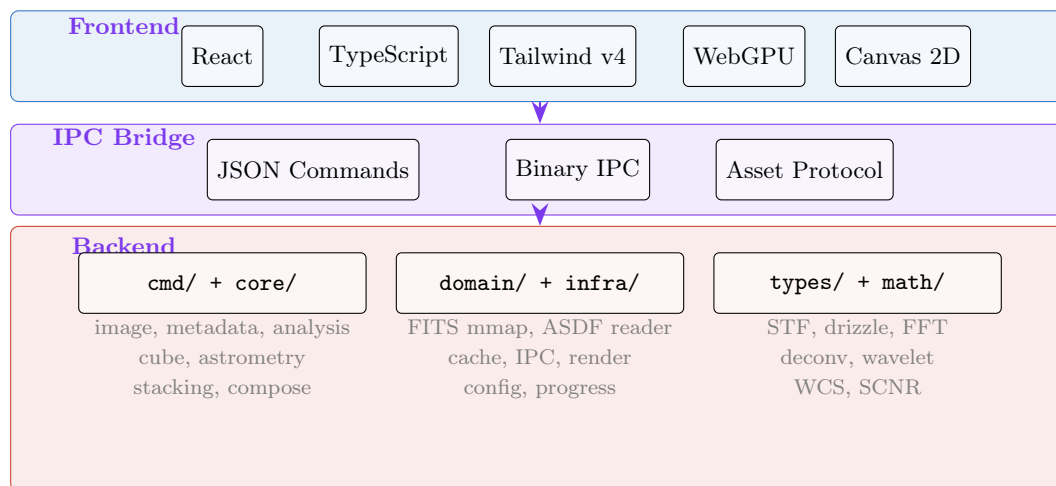


Figure 1: AstroBurst three-layer architecture

2.1 Backend (Rust)

The Rust backend is organized into six layers:

<code>cmd/</code>	Tauri command handlers organized by domain (41 registered commands). Each module receives deserialized JSON arguments, delegates to <code>core/</code> / <code>domain/</code> logic, and returns JSON or binary responses.
<code>core/</code>	Pure computational modules: analysis (histogram, FFT, stars), astrometry (plate solve, WCS), compose (RGB, SCNR, white balance), cube (spectrum, frames), imaging (STF, deconvolution, wavelet, background), metadata (header discovery, filter detection), stacking (sigma-clip, drizzle).
<code>domain/</code>	Orchestration layer coordinating core modules for complex multi-step operations (pipeline, batch).
<code>infra/</code>	Infrastructure: Arc-based zero-copy cache, config management, FITS I/O (mmap parser, MEF scanner, FITS writer), ASDF reader (YAML tree parser, binary block extractor, zlib/bzip2/lz4 decompressor, Roman data model converter, gWCS bridge), binary IPC encoding, progress tracking, PNG rendering.
<code>types/</code>	Shared type definitions: image stats, STF params, stacking/drizzle/RL configs, compose types, SCNR, error types, FITS header abstraction, constants.
<code>math/</code>	Mathematical primitives shared across core modules.

All heavy computation is offloaded via `tokio::task::spawn_blocking`, preventing UI freezes. Batch operations use Rayon for data-parallel execution across all available CPU cores.

2.2 Frontend (React + TypeScript)

The frontend is a single-page React application with a tab-based layout:

<code>App.tsx</code>	Root component managing view state (empty/processing/complete), file dialog integration, drag-and-drop, resizable sidebar, and folder scanning.
----------------------	---

useFileQueue

Central state machine (**useReducer**) tracking file status, concurrent processing queue (3 workers), resolution group detection, and auto-resampling.

useBackend

Typed Tauri IPC wrapper exposing all 41 commands with automatic URL resolution for `asset://` protocol paths and downsampled binary pixel transfer.

Tab System

Bottom tabs: Info (stats/header summary), Analysis (histogram, FFT, stars, spectroscopy), Headers (full header explorer), Export (FITS/PNG). Side tabs: Processing (background extraction, wavelet denoise, deconvolution), Compose (RGB, drizzle, drizzle RGB).

Preview Contexts

6 split React contexts (FileCtx, HistCtx, CubeCtx, RgbCtx, RenderCtx, RawPixelsCtx) providing scoped state to avoid cascading re-renders.

Lazy Loading

All panel components loaded via `React.lazy` + `Suspense` with spinner fallbacks.

2.3 IPC Strategy

Two IPC channels are used:

Binary IPC Protocol

The `get_raw_pixels_preview` and `get_raw_pixels_binary` commands encode a **16-byte header** followed by raw pixel data:

Offset	Type	Size	Description
0	u32	4B	Image width
4	u32	4B	Image height
8	f32	4B	Data minimum
12	f32	4B	Data maximum
16	f32[]	$W \times H \times 4B$	Raw pixel values

The frontend receives an `ArrayBuffer` directly with zero JSON/base64 overhead.

3 Mathematical Foundations

3.1 Screen Transfer Function (STF)

The STF implements the PixInsight-style **Midtone Transfer Function** (MTF). Given a normalized pixel value $x \in [0, 1]$ and midtone parameter $m \in (0, 1)$:

$$\text{MTF}(x, m) = \frac{(m - 1) \cdot x}{(2m - 1) \cdot x - m} \quad (1)$$

Properties:

- $\text{MTF}(0, m) = 0$ and $\text{MTF}(1, m) = 1$ for all m

- $\text{MTF}(m, m) = 0.5$ — the midtone maps to 50% brightness
- $m < 0.5$ brightens the image; $m > 0.5$ darkens it

The **auto-stretch** algorithm computes the stretch parameters from image statistics. Given the pixel distribution, let $\tilde{x}$ be the median and MAD the Median Absolute Deviation:

$$\text{MAD} = \text{median}(|x_i - \tilde{x}|) \quad (2)$$

$$\text{shadow} = \max(0, \tilde{x} - k \cdot \text{MAD}) \quad (3)$$

$$\text{midtone} = \text{MTF}^{-1}\left(t_{\text{bg}}, \frac{\tilde{x} - \text{shadow}}{1 - \text{shadow}}\right) \quad (4)$$

$$\text{highlight} = 1.0 \quad (5)$$

where $k = 2.8$ (calibrated for deep-sky data) and $t_{\text{bg}} = 0.25$ is the target background level. The clipping function is then:

$$\text{STF}(x) = \text{MTF}\left(\text{clip}\left(\frac{x - s}{h - s}, 0, 1\right), m\right) \quad (6)$$

where s, m, h are the shadow, midtone, and highlight parameters.

3.2 Asinh Stretch

The **arcsinh transfer function** provides astronomically-correct normalization that preserves flux ratios between stars of different magnitudes:

$$f(x) = \frac{\text{asinh}(\alpha \cdot x)}{\text{asinh}(\alpha)} \quad (7)$$

where α controls the stretch aggressiveness. This function maps $[0, 1] \rightarrow [0, 1]$ and is approximately linear for small x (preserving faint detail) while compressing bright values logarithmically.

3.3 Sigma-Clipped Stacking

Frames are registered via **cross-correlation** (phase correlation in frequency domain), then stacked with iterative sigma clipping.

Algorithm 1: Iterative Sigma-Clipped Stacking

Input : Aligned frames $\{F_1, F_2, \dots, F_N\}$, thresholds $\sigma_{\text{low}}, \sigma_{\text{high}}$, max iterations K
Output: Stacked image S

```

1 foreach pixel position  $(i, j)$  do
2    $V \leftarrow \{F_1(i, j), F_2(i, j), \dots, F_N(i, j)\};$ 
3   for  $k = 1$  to  $K$  do
4      $\mu \leftarrow \text{mean}(V);$ 
5      $\sigma \leftarrow \text{std}(V);$ 
6      $V' \leftarrow \{v \in V : \mu - \sigma_{\text{low}} \cdot \sigma \leq v \leq \mu + \sigma_{\text{high}} \cdot \sigma\};$ 
7     if  $|V'| = |V|$  or  $|V'| < 3$  then
8       break;
9     end
10     $V \leftarrow V';$ 
11  end
12   $S(i, j) \leftarrow \text{mean}(V);$ 
13 end

```

Default parameters: $\sigma_{\text{low}} = \sigma_{\text{high}} = 3.0$, $K = 5$.

3.4 Drizzle Integration

Drizzle (Variable-Pixel Linear Reconstruction) maps input pixels onto a higher-resolution output grid. For an input pixel p at fractional position (x_f, y_f) on the output grid:

$$O(i, j) = \frac{\sum_p w_p \cdot K(i - x_f^p, j - y_f^p) \cdot I_p}{\sum_p w_p \cdot K(i - x_f^p, j - y_f^p)} \quad (8)$$

where K is the drizzle kernel and w_p is the pixel weight. Three kernel options are implemented:

$$K_{\text{square}}(\Delta x, \Delta y) = \begin{cases} 1 & \text{if } |\Delta x| \leq \frac{d}{2} \text{ and } |\Delta y| \leq \frac{d}{2} \\ 0 & \text{otherwise} \end{cases} \quad (9)$$

$$K_{\text{gaussian}}(\Delta x, \Delta y) = \exp\left(-\frac{\Delta x^2 + \Delta y^2}{2\sigma_K^2}\right) \quad (10)$$

$$K_{\text{lanczos3}}(\Delta x, \Delta y) = L_3(\Delta x) \cdot L_3(\Delta y) \quad (11)$$

where $d = \text{pixfrac}/\text{scale}$ and the Lanczos-3 kernel is:

$$L_3(x) = \begin{cases} \text{sinc}(x) \cdot \text{sinc}(x/3) & \text{if } |x| < 3 \\ 0 & \text{otherwise} \end{cases} \quad (12)$$

The weight map $W(i, j) = \sum_p w_p \cdot K(\cdot)$ tracks per-pixel coverage for noise estimation.

3.5 Cross-Correlation Alignment

Frame registration uses **phase correlation** in the frequency domain. Given reference frame R and target frame T :

$$\text{shift} = \arg \max_{(\Delta x, \Delta y)} \mathcal{F}^{-1} \left\{ \frac{\hat{R} \cdot \hat{T}^*}{|\hat{R} \cdot \hat{T}^*|} \right\} \quad (13)$$

where $\hat{R} = \mathcal{F}\{R\}$ denotes the 2D DFT. The normalized cross-power spectrum produces a sharp peak at the integer translation offset.

3.6 World Coordinate System (WCS)

WCS transforms use the **CD matrix formulation** with TAN (gnomonic) projection. Given reference pixel (CRPIX₁, CRPIX₂) and reference coordinate (α_0, δ_0) = (CRVAL₁, CRVAL₂):

3.6.1 Forward Transform (Pixel → World)

$$\begin{pmatrix} u \\ v \end{pmatrix} = \begin{pmatrix} \text{CD}_{1,1} & \text{CD}_{1,2} \\ \text{CD}_{2,1} & \text{CD}_{2,2} \end{pmatrix} \begin{pmatrix} x - \text{CRPIX}_1 \\ y - \text{CRPIX}_2 \end{pmatrix} \quad (14)$$

where (u, v) are intermediate world coordinates in degrees. The TAN deprojection yields:

$$\alpha = \alpha_0 + \arctan \left(\frac{u}{\cos(\delta_0) - v \cdot \sin(\delta_0)} \right) \quad (15)$$

$$\delta = \arctan \left(\frac{v \cdot \cos(\delta_0) + \sin(\delta_0)}{\sqrt{u^2 + (\cos(\delta_0) - v \cdot \sin(\delta_0))^2}} \right) \quad (16)$$

3.6.2 Pixel Scale

$$\text{scale} = \sqrt{|\det(\mathbf{CD})|} \times 3600 \quad [\text{arcsec/pixel}] \quad (17)$$

3.6.3 Inverse Transform (World → Pixel)

$$\begin{pmatrix} x \\ y \end{pmatrix} = \mathbf{CD}^{-1} \begin{pmatrix} u \\ v \end{pmatrix} + \begin{pmatrix} \text{CRPIX}_1 \\ \text{CRPIX}_2 \end{pmatrix} \quad (18)$$

3.7 FFT Power Spectrum

The 2D power spectrum for noise characterization is computed as:

$$P(u, v) = \log_{10} \left(1 + |\hat{I}(u, v)|^2 \right) \quad (19)$$

where $\hat{I} = \mathcal{F}\{I\}$ is the 2D DFT of the image. The DC component at (0,0) is shifted to the center for visualization. The spectrum is normalized to [0, 1] after computation.

3.8 Star Detection (PSF Centroid)

Background is estimated via iterative sigma-clipped median. Pixels above $\tilde{b} + \sigma_{\text{thresh}} \cdot \sigma_b$ are identified as candidates. For each connected component C :

$$\bar{x}_C = \frac{\sum_{(i,j) \in C} i \cdot (I_{ij} - b)}{\sum_{(i,j) \in C} (I_{ij} - b)} \quad \bar{y}_C = \frac{\sum_{(i,j) \in C} j \cdot (I_{ij} - b)}{\sum_{(i,j) \in C} (I_{ij} - b)} \quad (20)$$

$$F_C = \sum_{(i,j) \in C} (I_{ij} - b) \quad \text{SNR}_C = \frac{F_C}{\sqrt{|C| \cdot \sigma_b^2}} \quad (21)$$

FWHM is estimated from the second intensity moments:

$$\text{FWHM} = 2\sqrt{2 \ln 2} \cdot \sqrt{\frac{\sigma_x^2 + \sigma_y^2}{2}} \quad (22)$$

where σ_x^2, σ_y^2 are the variance of pixel positions weighted by intensity.

3.9 SCNR (Subtractive Chromatic Noise Reduction)

SCNR removes green channel excess in narrowband compositions. Two methods are implemented:

$$G'_{\text{average}} = \min\left(G, \frac{R+B}{2}\right) \quad (23)$$

$$G'_{\text{max}} = \min(G, \max(R, B)) \quad (24)$$

3.10 Calibration Pipeline

The standard CCD calibration formula applies bias, dark current, and flat field corrections:

$$I_{\text{calibrated}} = \frac{I_{\text{raw}} - M_{\text{bias}} - r \cdot M_{\text{dark}}}{M_{\text{flat}} / \overline{M_{\text{flat}}}} \quad (25)$$

where M_{bias} , M_{dark} , M_{flat} are master calibration frames (median-combined), r is the exposure ratio (science/dark), and $\overline{M_{\text{flat}}}$ is the mean value of the flat field.

3.11 White Balance

Auto white balance normalizes each channel by its median value:

$$C'_k = \frac{C_k}{\tilde{C}_k} \cdot \min(\tilde{C}_R, \tilde{C}_G, \tilde{C}_B), \quad k \in \{R, G, B\} \quad (26)$$

3.12 Richardson-Lucy Deconvolution (v0.3.0)

The Richardson-Lucy algorithm iteratively estimates the true image f given observed image g and point spread function h :

$$f^{(k+1)} = f^{(k)} \cdot \left(\frac{g}{f^{(k)} * h} * \hat{h} \right) \quad (27)$$

where $*$ denotes convolution, $\hat{h}$ is the PSF flipped about both axes, and $f^{(0)} = g$. Convolutions are computed in the frequency domain via FFT for efficiency:

$$f^{(k)} * h = \mathcal{F}^{-1} \left\{ \hat{F}^{(k)} \cdot \hat{H} \right\} \quad (28)$$

where $\hat{F}^{(k)} = \mathcal{F}\{f^{(k)}\}$ and $\hat{H} = \mathcal{F}\{h\}$. The PSF is modeled as a 2D Gaussian:

$$h(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right) \quad (29)$$

with configurable σ (default 2.0) and kernel size (default 15×15). Tikhonov regularization is applied:

$$f_{\text{reg}}^{(k+1)} = \frac{f^{(k+1)}}{1 + \lambda \nabla^2 f^{(k+1)}} \quad (30)$$

where λ is the regularization parameter (default 0.001). Optional deringing suppresses ringing artifacts near bright edges by clamping values that exceed a threshold above the local median.

3.13 Background Extraction (v0.3.0)

Background is modeled as a polynomial surface fitted to sigma-clipped grid samples:

1. Divide the image into an $N \times N$ grid (default $N = 8$)
2. For each cell, compute the sigma-clipped median (default $\sigma = 2.5$, 3 iterations) to reject stars and nebulosity
3. Fit a 2D polynomial of degree d (default $d = 3$) to the grid medians using least squares:

$$B(x, y) = \sum_{i=0}^d \sum_{j=0}^{d-i} a_{ij} x^i y^j \quad (31)$$

Two correction modes are supported:

$$I_{\text{corrected}} = \begin{cases} I - B & \text{(subtract mode)} \\ I/B & \text{(divide mode)} \end{cases} \quad (32)$$

3.14 Wavelet Denoising (v0.3.0)

Multi-scale wavelet denoising uses the à trous (“with holes”) algorithm. The image is decomposed into N wavelet scales W_k and a residual c_N :

$$I = \sum_{k=1}^N W_k + c_N \quad (33)$$

where each wavelet scale $W_k = c_{k-1} - c_k$ captures structures at spatial frequency 2^k pixels. The smoothed approximation c_k is obtained by convolving c_{k-1} with a B3 spline kernel dilated by factor 2^{k-1} .

Denoising applies a hard threshold to each scale:

$$W'_k(x, y) = \begin{cases} W_k(x, y) & \text{if } |W_k(x, y)| > \sigma_k \cdot t_k \\ 0 & \text{otherwise} \end{cases} \quad (34)$$

where σ_k is the noise estimate at scale k (from the MAD of the wavelet coefficients) and t_k is the per-scale threshold (default: 3.0, 2.5, 2.0, 1.5, 1.0 for scales 1–5). The denoised image is reconstructed as:

$$I' = \sum_{k=1}^N W'_k + c_N \quad (35)$$

Up to 8 decomposition scales are supported (controlled by the `numScales` parameter).

4 Command Module Reference

The backend exposes **41 Tauri commands** across 9 modules, registered in `lib.rs` via `tauri::generate_handler`. Of these, **20 are actively used** by the frontend UI and **21 are backend-ready** but awaiting UI integration.

Command Wiring Status

Commands marked with **★** are actively wired to UI components. Commands marked with **○** have working backend implementations but no frontend panel yet.

4.1 Image Module (commands/image.rs)

Command	Parameters	Returns
★ <code>process_fits</code>	path, output_dir	png_path, dimensions, elapsed_ms
★ <code>process_fits_full</code>	path, output_dir	png_path, dims, header, histogram, stf
○ <code>process_batch</code>	paths[], output_dir	processed, failed, results[]
○ <code>get_raw_pixels_binary</code>	path	ArrayBuffer (16B header + f32[])
★ <code>get_raw_pixels_preview</code>	path, max_dim	ArrayBuffer (downsampled, 16B header + f32[])
★ <code>export_fits</code>	path, output_path, stf opts, copy_wcs?	output_path, dims, file_size
★ <code>export_fits_rgb</code>	r/g/b_path, output_path, copy_wcs?	output_path, dimensions
★ <code>resample_fits_cmd</code>	path, target_w, target_h, output_dir	png_path, fits_path, dims

Table 1: Image module commands

4.2 Metadata Module (commands/metadata.rs)

Command	Parameters	Returns
★ <code>get_header</code>	path	Map of 20 key FITS keywords
★ <code>get_full_header</code>	path	All cards categorized, filter detection
○ <code>get_fits_extensions</code>	path	HDU list with types and dimensions
○ <code>get_header_by_hdu</code>	path, hdu_index	Header cards for specific HDU
○ <code>detect_narrowband_filters</code>	paths[]	Palette suggestion with channel mapping

Table 2: Metadata module commands

4.3 Analysis Module (commands/analysis.rs)

Command	Parameters	Returns
★ <code>compute_histogram</code>	path	65536 bins, stats (median/mean/ σ /MAD), auto_stf
★ <code>compute_fft_spectrum</code>	path, output_dir	width, height, pixels_b64, dc/-max magnitude, png_path
★ <code>detect_stars</code>	path, sigma?, max_stars?	stars[] (x,y,flux,fwhm,snr), background

Table 3: Analysis module commands

4.4 Visualization Module (commands/visualization.rs)

Command	Parameters	Returns
* <code>apply_stf_render</code>	path, output_dir, shadow, midtone, highlight	png_path, dims, stf_params
o <code>generate_tiles</code>	path, output_dir, tile_size?	tile_size, levels[], original dims
o <code>get_tile</code>	path, output_dir, level, col, row	tile_path, cached flag

Table 4: Visualization module commands

4.5 Cube Module (commands/cube.rs)

Command	Parameters	Returns
o <code>process_cube_cmd</code>	path, output_dir, frame_step?	dims, collapsed paths, spectrum
o <code>process_cube_lazy_cmd</code>	path, output_dir, frame_step?	Same + total_frames (streaming)
* <code>get_cube_info</code>	path	naxis1/2/3, bitpix, wavelengths
o <code>get_cube_frame</code>	path, frame_index, output_path	frame_index, output_path, elapsed
* <code>get_cube_spectrum</code>	path, x, y	spectrum[], wavelengths[], elapsed

Table 5: Cube module commands

4.6 Astrometry Module (commands/astrometry.rs)

Command	Parameters	Returns
o <code>plate_solve_cmd</code>	path, sigma?, hints?, scale range?	ra/dec center, pixel_scale, WCS matrix
o <code>get_wcs_info</code>	path	center coords, pixel_scale, FOV, corners
o <code>pixel_to_world</code>	path, x, y	ra, dec, ra_dec_str
o <code>world_to_pixel</code>	path, ra, dec	x, y

Table 6: Astrometry module commands

4.7 Stacking Module (commands/stacking.rs)

Command	Parameters	Returns
o <code>calibrate</code>	science, output_dir, bias/dark/flat?, ratio?	png_path, dims, applied flags
o <code>stack</code>	paths[], output_dir, σ params, align?	png_path, count, rejected, offsets
* <code>drizzle_stack_cmd</code>	paths[], output_dir, scale, pixfrac, kernel	png, weight_map, dims, offsets
* <code>drizzle_rgb_cmd</code>	r/g/b groups, output_dir, drizzle params	png_path, per-channel info
* <code>compose_rgb_cmd</code>	r/g/b_path, output_dir, stretch/wb/scnr	png_path, per-channel STF, offsets
o <code>run_pipeline_cmd</code>	input, output_dir, frame_step?	total, succeeded, failed, results

Table 7: Stacking module commands

4.8 Config Module (commands/config.rs)

Command	Parameters	Returns
◦ <code>get_config</code>	(none)	<code>has_api_key</code> , <code>urls</code> , <code>defaults</code> , <code>stretch params</code>
◦ <code>update_config</code>	<code>field</code> , <code>value</code>	<code>{ updated: true }</code>
◦ <code>save_api_key</code>	<code>key</code> , <code>service?</code>	<code>{ saved: true, service }</code>
◦ <code>get_api_key</code>	(none)	<code>is_set</code> , <code>masked key</code>

Table 8: Config module commands

4.9 Processing Module (commands/processing.rs) — New in v0.3.0

Command	Parameters	Returns
★ <code>deconvolve_rl_cmd</code>	<code>path</code> , <code>output_dir</code> , <code>iterations</code> , <code>psf_sigma</code> , <code>psf_size</code> , <code>regularization</code> , <code>deringing</code> , <code>deringing_threshold</code>	<code>png_path</code> , <code>iterations_run</code> , <code>convergence</code> , <code>elapsed</code>
★ <code>extract_background_cmd</code>	<code>path</code> , <code>output_dir</code> , <code>grid_size</code> , <code>poly_degree</code> , <code>sigma_clip</code> , <code>iterations</code> , <code>mode</code>	<code>corrected_png</code> , <code>model_png</code> , <code>stats</code>
★ <code>wavelet_denoise_cmd</code>	<code>path</code> , <code>output_dir</code> , <code>num_scales</code> , <code>thresholds[]</code> , <code>linear</code>	<code>png_path</code> , <code>scales_applied</code> , <code>elapsed</code>

Table 9: Processing module commands (v0.3.0)

5 FITS I/O Implementation

5.1 Memory-Mapped Extraction

FITS files are opened via `mmap2`, avoiding loading entire files into memory. The extraction pipeline:

1. Parse header sequentially in 2880-byte blocks until `END` keyword
2. Compute data offset: $\text{offset} = \lceil \text{header_bytes} / 2880 \rceil \times 2880$
3. Memory-map the data region as a byte slice
4. BITPIX-dependent byte-swapping from FITS big-endian to native `f32`

For integer data (`BITPIX > 0`), the `BSCALE/BZERO` transform is applied:

$$\text{value}_{\text{physical}} = \text{BSCALE} \times \text{value}_{\text{stored}} + \text{BZERO} \quad (36)$$

5.2 Supported BITPIX Values

BITPIX	Type	Bytes/px	Conversion
8	Unsigned 8-bit int	1	Direct cast + BSCALE/BZERO
16	Signed 16-bit int	2	Big-endian swap + BSCALE/BZERO
32	Signed 32-bit int	4	Big-endian swap + BSCALE/BZERO
-32	IEEE 754 float	4	Big-endian swap
-64	IEEE 754 double	8	Big-endian swap → f32

Table 10: FITS BITPIX type handling

6 ASDF I/O Implementation (v0.3.0)

AstroBurst includes the first Rust implementation of the **Advanced Scientific Data Format** (ASDF) reader, targeting NASA’s Roman Space Telescope data products.

6.1 Format Structure

An ASDF file consists of three sections:

1. **YAML tree** — Metadata, data model, and WCS information serialized as YAML
2. **Binary blocks** — N-dimensional array data with optional compression
3. **Block index** — Byte offsets for random access to blocks

6.2 Decompression Support

Binary blocks may be compressed with zlib (`flate2`), bzip2 (`bzip2`), or lz4 (`lz4_flex`). The reader auto-detects the compression codec from the block header and decompresses transparently.

6.3 Roman Data Model Traversal

Roman Space Telescope ASDF files use a nested data model. The reader traverses the YAML tree to locate the science data array at `roman.data` or `roman.meta.exposure.data`, extracts the associated `ndarray` block reference, and converts it to `Array2<f32>` via the `AsdfImage` bridge.

6.4 gWCS Extraction

The generalized World Coordinate System (gWCS) stored in `roman.meta.wcs` is parsed to extract celestial reference coordinates and pixel scale, enabling coordinate overlay and astrometric context for Roman data.

6.5 Auto-Dispatch

The `extract_image_resolved` and `load_cached` functions auto-detect file format by extension (`.asdf` vs `.fits/.fit/.fts`) and dispatch to the appropriate reader. The frontend accepts both formats transparently via the same drag-and-drop and file picker interface.

7 WebGPU Compute Pipeline

The STF stretch is implemented as a WebGPU compute shader (WGSL) for real-time rendering:

STF Compute Shader (simplified WGSL)

```
@group(0) @binding(0) var<storage, read> input: array<f32>;
@group(0) @binding(1) var<storage, read_write> output: array<u32>;
@group(0) @binding(2) var<uniform> params: StfParams;

fn mtf(x: f32, m: f32) -> f32 {
    return (m - 1.0) * x / ((2.0 * m - 1.0) * x - m);
}

@compute @workgroup_size(256)
fn main(@builtin(global_invocation_id) id: vec3<u32>) {
    let idx = id.x;
    let v = (input[idx] - params.shadow) / (params.highlight - params.shadow);
    let stretched = mtf(clamp(v, 0.0, 1.0), params.midtone);
    let byte_val = u32(clamp(stretched * 255.0, 0.0, 255.0));
    output[idx] = (255u << 24u) | (byte_val << 16u) | (byte_val << 8u) | byte_val;
}
```

The shader processes pixels in parallel with workgroup size 256. Falls back to Canvas 2D `putImageData` when WebGPU is unavailable (Chromium < 113).

8 Performance Benchmarks

Operation	Input Size	Time	Notes
Single FITS processing	4096×4096 (64 MB)	120 ms	533 MB/s throughput
Batch (10 frames)	10×64 MB	450 ms	1.4 GB/s (Rayon parallel)
Histogram + auto-STF	4096×4096	35 ms	SIMD-accelerated
Star detection ($\sigma=5$)	4096×4096	80 ms	~3000 stars
Sigma-clip stack	10×64 MB	2.1 s	5 iterations
Drizzle 2×	10×64 MB	4.8 s	Output: 8192×8192
FFT power spectrum	4096×4096	95 ms	<code>rustfft</code>
Cube spectrum extraction	500 ² ×2000	12 ms	Memory-mapped
WebGPU STF render	4096×4096	8 ms	GPU compute
Open 2 GB IFU cube	2 GB datacube	300 ms	<code>mmap2</code> lazy
Drizzle RGB 2×	3×10×64 MB	6.8 s	All channels
RGB compose + align	3 channels	1.8 s	With cross-correlation

Table 11: Performance benchmarks (AMD Ryzen 9 7950X, 64 GB DDR5, NVMe)

Binary size: ~15 MB.

9 Dependencies

9.1 Rust Crates

Crate	Version	Purpose
tauri	2.x	Application framework, IPC, window management
ndarray	0.16	N-dimensional arrays, Rayon parallel iterators
rayon	1.10	Data-parallel batch processing
memmap2	0.9	Memory-mapped file I/O for FITS
rustfft	6.2	FFT for deconvolution and power spectrum
num-complex	0.4	Complex number support for FFT
image	0.25	PNG encoding/decoding
serde_yaml	0.9	ASDF YAML tree parsing
flate2	1.0	zlib decompression (ASDF blocks)
bzip2	0.4	bzip2 decompression (ASDF blocks)
lz4_flex	0.11	lz4 decompression (ASDF blocks)
request	0.12	HTTP client for astrometry.net (optional)
zip	0.6	ZIP archive extraction for compressed FITS
serde / serde_json	1.x	Serialization for Tauri commands
anyhow / thiserror	1.0+	Error handling with context and typed errors
base64	0.22	Binary-to-text encoding for JSON IPC
regex	1.10	Filter detection pattern matching
lazy_static	1.5	Static initialization
dirs	5.0	OS-native directory resolution
tempfile	3.8	Temporary files for ZIP extraction

Table 12: Rust dependencies

9.2 Frontend

Package	Purpose
react / react-dom	UI framework
@tauri-apps/api + plugins	Tauri IPC, dialog, filesystem
framer-motion	Animations and transitions
lucide-react	Icon system
jszip / file-saver	Client-side ZIP export
tailwindcss v4	Utility-first CSS
vite	Build toolchain
typescript	Type safety

Table 13: Frontend dependencies

10 Security Considerations

- API keys (astrometry.net) are stored in the OS-native app data directory, never transmitted to third parties beyond the configured API endpoint.
- All file system access is constrained by Tauri capabilities (`default.json`). Permissions are scoped to home, desktop, and appdata directories.
- No telemetry, analytics, or network calls are made except for explicit plate solving requests initiated by the user.
- FITS and ASDF files are processed locally; no data leaves the machine unless plate solving is triggered.

11 Known Limitations

1. **21 unwired commands** — Plate solving, calibration, sigma-clip stacking, tile deep zoom, WCS overlay, narrowband auto-detection, and config UI panels are backend-ready but lack frontend integration
2. **Grayscale pipeline** — Color FITS images require manual RGB channel assignment
3. **Online plate solving only** — Local index-based solving is planned
4. **No undo/redo** — Operations are destructive (original files preserved)
5. **WebGPU requirement** — Chromium 113+ needed; Canvas 2D fallback available
6. **No session persistence** — Reopening the app starts a fresh workspace
7. **ASDF read-only** — Writer not yet implemented; ASDF files can be loaded but not exported
8. **No MAST API integration** — JWST/Roman data must be downloaded manually

12 Roadmap

Feature	Priority	Status
Multi-extension FITS (MEF)	High	Done (v0.2)
Bicubic resampling	High	Done (v0.2)
Richardson-Lucy deconvolution	High	Done (v0.3)
Background extraction	High	Done (v0.3)
Wavelet noise reduction	High	Done (v0.3)
ASDF format support	High	Done (v0.3)
Wire 21 backend commands to UI	High	Planned (v0.4)
Local plate solving	High	Planned
Star removal	Medium	Planned (v0.5)
Photometric calibration (Gaia DR3)	Medium	Planned (v0.5)
PixelMath expressions	Medium	Planned (v0.6)
Mosaic stitching	Medium	Planned (v0.6)
Plugin architecture (WASM)	Medium	Design phase
Undo/redo system	Medium	Planned
MAST API integration	Low	Planned
Live stacking mode	Low	Planned
INDI/ASCOM integration	Low	Planned

Table 14: Development roadmap

A Sample Test Data

The repository includes HST/WFPC2 narrowband FITS images in `tests/sample-data/`:

File	Filter	λ	Hubble Palette	Description
502nmos.fits	[OIII]	502 nm	Blue channel	Oxygen III emission
656nmos.fits	H α	656 nm	Green channel	Hydrogen alpha emission
673nmos.fits	[SII]	673 nm	Red channel	Sulfur II emission

Table 15: Sample FITS files — Eagle Nebula (M16) region, 1600×1600 float32

All images: BITPIX = -32 , WCS TAN projection, origin STScI-STSDAS. Public domain (NASA/ESA).